

ExamBuddy (SmartStudy)

Complete Technical Documentation

Codebase Technical Audit Report

Audit date: 19 August 2026

Scope: Verified from source code only

Constraint: No application code was modified

Source markdown: EXAMBUDDY_TECHNICAL_DOCUMENTATION.md

If something could not be verified from the codebase, the markdown states: Not found in the current codebase.

1. PROJECT OVERVIEW

Simple explanation

ExamBuddy is a student exam-prep mobile app. A student:

1. Creates an account and logs in.
2. Uploads previous-year question papers (PYQs) and optionally a syllabus PDF.
3. The backend extracts text, finds topics (mostly with rules), and may call an LLM to enrich analysis.
4. The student browses subjects -> modules -> topics, sees exam priority from PYQ frequency, and generates AI study notes and quizzes.

Technical explanation

ExamBuddy is a client-server system:

- Frontend: Expo / React Native mobile app (mobile/).
- Backend: FastAPI REST API (backend/), versioned under /api/v1.
- Database: MongoDB via Motor async driver (database name smartstudy).
- AI: Multi-provider LLM layer (Groq / Google Gemini / OpenAI) with local rule-based fallbacks.
- Files: Cloudinary when configured; otherwise local backend/uploads/.
- "RAG": Keyword/term scoring over stored document text chunks - not vector embeddings / vector DB.

Main purpose

Help engineering/university students prepare from syllabus structure + PYQ occurrence, then generate topic notes and quizzes using LLMs grounded (when possible) in uploaded materials.

Main user workflow (verified)

Login -> Upload PYQ / Syllabus -> Process documents -> Run PYQ analysis -> Browse Notes by Subject -> Module -> Topic -> Generate notes -> Take quiz.

Major features currently implemented

Feature	Evidence
Auth (register/login/JWT/refresh/forgot-reset)	backend/app/api/v1/auth.py, auth_service.py, security.py
Document upload (PDF/DOCX/images)	documents.py, document_service.py, file_service.py
Text extraction + OCR (optional)	text_extractor.py, PyMuPDF, pytesseract
Syllabus structure parse + module tree	syllabus_parser.py, syllabus_modules.py
PYQ analysis pipeline	analysis_service.py, ai_service.analyze_pyq, topic_extractor.py
Topic priority / occurrence	topic_priority.py, subjects overview API
Topic notes generation + cache	notes_service.py, generated_notes collection
Quiz generate / submit / history	quiz.py, quiz services/repos
Profile / dashboard	profile.py
Mobile UI for all of the above	mobile/src/presentation/screens/*, navigators

Major technologies used

FastAPI, Uvicorn, Motor/PyMongo, MongoDB, JWT (python-jose), bcrypt/passlib, Cloudinary (optional), PyMuPDF, python-docx, Pillow, pytesseract, Groq (OpenAI-compatible client), Google Generative AI, OpenAI SDK, Expo/React Native, Zustand, axios, Expo SecureStore, React Navigation, EAS/Render Docker deploy configs.

2. COMPLETE TECHNOLOGY STACK

Layer	Technology	Where Used	Evidence/File
Frontend	Expo ~54, React Native 0.81, React 19.1	Mobile app	mobile/package.json
Frontend UI	react-native-paper, vector icons	Screens/components	mobile/package.json, screens
Frontend nav	React Navigation (stack + tabs)	App navigation	RootNavigator.tsx, MainTabNavigator.tsx, AuthNavigator.tsx
Frontend state	Zustand	Auth/notes/quiz stores	mobile/src/store/*
Frontend HTTP	axios	API client	mobile/package.json, API modules
Backend	FastAPI 0.115, Uvicorn	REST API	backend/requirements.txt, backend/app/main.py
Validation	Pydantic v2, pydantic-settings	Schemas + Settings	backend/app/core/config.py, schemas/
Database	MongoDB + Motor 3.7 / PyMongo 4.11	Persistence	backend/app/db/mongodb.py, repositories
Authentication	JWT HS256 + bcrypt	Auth	backend/app/core/security.py, .env.example
AI/LLM	Groq, Gemini, OpenAI (configurable)	Analysis, notes, quiz	ai_service.py, base_provider.py, config.py
File Storage	Cloudinary or local disk	Uploads	file_service.py
Vector Database	Not found in the current codebase	-	No FAISS/Chroma/Pinecone/pgvector
Embeddings	Not found in the current codebase	-	No embedding model/API usage for retrieval
OCR	pytesseract + Pillow (optional path)	Image/PDF OCR	text_extractor.py, requirements.txt
PDF text	PyMuPDF (fitz)	PDF extraction	text_extractor.py, requirements.txt
DOCX	python-docx	DOCX extraction	text_extractor.py
PDF export	fpdf2	Notes PDF download	requirements.txt, notes export endpoint
Deployment (API)	Docker + Render blueprint	Backend hosting	backend/Dockerfile, render.yaml
Deployment (mobile)	EAS Build profiles	APK builds	mobile/eas.json, package.json scripts
Email	SMTP settings (optional)	Password reset	config.py, .env.example

3. LLM / AI MODEL AUDIT

Provider architecture (verified)

AIService (backend/app/services/ai/ai_service.py) builds providers from settings and tries them with fallback via _generate_json_with_fallback.

Configured defaults (backend/app/core/config.py / .env.example):

Provider	Setting keys	Default model name
Groq	AI_PROVIDER=groq, GROQ_API_KEY, GROQ_MODEL	llama-3.3-70b-versatile

Gemini	GEMINI_API_KEY, GEMINI_MODEL	gemini-2.5-flash
OpenAI	OPENAI_API_KEY, OPENAI_MODEL	gpt-4o-mini

Production Render blueprint (render.yaml) sets AI_PROVIDER=gemini and GEMINI_MODEL=gemini-2.5-flash. Which provider actually runs in a given environment depends on env vars present at runtime - not hard-coded to a single live key in the repo.

Anthropic / Claude: Not found as an implemented provider in the current codebase.

Shared call mechanics

File: backend/app/services/ai/base_provider.py

- JSON mode supported.
- Default temperatures (when not overridden): ~0.3-0.4 depending on provider/json mode.
- max_tokens / maxOutputTokens configured per call path.
- Fallback across configured providers when one fails.

Feature: PYQ Analysis enrichment

Field	Value
Feature	PYQ analysis enrichment / topic frequency JSON
Provider	Primary from AI_PROVIDER, then fallbacks with keys
Model	Env-configured (GROQ_MODEL / GEMINI_MODEL / OPENAI_MODEL)
API	Provider HTTP APIs via Groq/OpenAI-compatible client or google-generativeai
File	backend/app/services/ai/ai_service.py
Function	AIService.analyze_pyq
Input	Cleaned PYQ content + pre-extracted topics + subject
Output	Structured JSON (topic frequency tables, priorities, summary fields)
Prompt	PYQ_ANALYSIS_SYSTEM_PROMPT, PYQ_ANALYSIS_USER_PROMPT in prompts.py
Structured output	Yes - JSON
Temperature	Provider defaults in base_provider.py
Error handling	Falls back to local/rule-based result (provider: local, model: rule-based)
Fallback model	Other configured LLM providers, then local

Feature: Topic Notes Generation

Field	Value
Feature	Per-topic study notes
Provider / Model	Same multi-provider stack
File	ai_service.py -> generate_topic_notes / stream_topic_notes
Called from	NotesService.generate_topic_note
Input	Subject, topic, exam priority hint, related PYQs, RAG context, analysis context, pipeline context
Output	Structured notes JSON (definition, explanation, FAQs, etc.)
Prompt	TOPIC_NOTES_SYSTEM_PROMPT, TOPIC_NOTES_USER_PROMPT (PROMPT_VERSION = "v15.0")

Structured output	Yes - JSON
Error handling	Local topic notes fallback (_local_topic_notes)
Caching	generated_notes collection keyed by user/topic/analysis + prompt version

Feature: Batch Notes Generation

Field	Value
Feature	Batch notes from analysis topics
File / Function	AIService.generate_notes
Prompt	NOTES_GENERATE_SYSTEM_PROMPT, NOTES_GENERATE_USER_PROMPT
Structured output	Yes - JSON with topic_notes[]

Feature: Notes Simplify

Field	Value
Feature	Simplify existing notes
File / Function	AIService.simplify_notes
Prompt	NOTES_SIMPLIFY_*

Feature: Quiz Generation

Field	Value
Feature	Quiz questions
File / Function	AIService.generate_quiz (via QuizService)
Prompt	QUIZ_GENERATE_SYSTEM_PROMPT, QUIZ_GENERATE_USER_PROMPT
Input	num_questions, quiz_type, difficulty, subject, topics, content (capped ~30,000 chars)
Output	JSON questions with options, correct_answer, explanation, topic
Fallback	_local_quiz_result rule templates

On-device mobile LLM SDKs

Not found in the current codebase (mobile/package.json has no LLM SDKs). All AI calls go through the backend API.

4. AI FEATURES (INVENTORY)

4.1 PYQ analysis (hybrid)

```

Uploaded PYQ docs
-> extracted_text (already stored)
-> NotesPipeline / cleaning + rule topic extraction (topic_extractor)
-> LLM analyze_pyq (enrich/normalize JSON) OR local fallback
-> optional syllabus match (match_topics_to_syllabus)
-> pyq_analyses MongoDB
-> AnalysisResult screen (mobile)

```

4.2 Topic extraction (rule-based primary)

Cleaned question text

- > regex/heuristic noun phrases, filters (topic_extractor.py)
- > frequency Counter
- > LLM may validate/merge names during analyze_pyq

4.3 Syllabus analysis (rule-based)

Syllabus PDF/DOCX

- > text extraction
- > extract_syllabus_structure (regex heuristics)
- > stored on document.syllabus_structure
- > later build_module_topic_tree for Notes UI

LLM involvement in syllabus parsing: Not found (heuristic parser only).

4.4 Topic classification / importance

- Rule + relative scoring in topic_priority.py (High/Medium/Low).
- LLM analysis JSON also contains priority-like fields; post-processing sanitizes via sanitize_analysis_result / priority utils.

4.5 Notes generation (LLM + keyword retrieval context)

User selects topic

- > NotesService cache check
- > DocumentRetriever keyword RAG context
- > AIService.generate_topic_notes
- > upsert generated_notes
- > TopicStudyNotes UI

4.6 Quiz generation (LLM)

QuizConfig selections

- > QuizService.generate_quiz
- > AIService.generate_quiz
- > quizzes collection
- > QuizPlay / submit -> quiz_attempts

4.7 Question generation (standalone)

Covered by quiz generation. Separate "question bank" product feature beyond quizzes: Not found as a distinct module.

4.8 Summarization

Present as notes fields (summary) and simplify-notes flow - not a standalone summarizer product feature beyond that.

4.9 Difficulty classification of PYQ questions

Dedicated ML difficulty classifier: Not found. Quiz difficulty is a user-selected parameter passed into the quiz prompt.

5. RAG PIPELINE AUDIT

Verdict

The app implements a retrieval-augmented notes context path, but it is keyword/term scoring over

MongoDB-stored text chunks, not a vector embedding RAG stack.

Searches for FAISS, Chroma, Pinecone, Weaviate, Qdrant, pgvector, Elasticsearch, LangChain, LlamaIndex, embedding APIs: Not found as implemented retrieval infrastructure.

Actual pipeline (verified)

Document upload

```
-> extract_text (PyMuPDF / DOCX / OCR)
-> preprocess_pyq_text
-> build_text_chunks (size 1200, overlap 150; max stored ~80)
-> store extracted_text + text_chunks on documents
    v
```

Notes generation request

```
-> DocumentRetriever (rag/retriever.py)
-> load user docs (prefer analysis-linked)
-> score chunks by topic terms + category boost
-> select top chunks (max ~8, ~10k chars)
-> sanitize_rag_passage
-> inject into TOPIC_NOTES_USER_PROMPT as {rag_context}
-> LLM
-> notes JSON
```

Step	File	Function / detail
Text extraction	text_extractor.py	extract_text
Chunking (store)	pdf_processor.py	build_text_chunks, RAG_CHUNK_SIZE=1200
Vector embedding	-	Not found
Vector storage	-	Not found
Query embedding	-	Not found
Similarity search	rag/retriever.py	_score_chunk term/keyword scoring
Prompt construction	prompts.py + notes service	{rag_context}
LLM	ai_service.generate_topic_notes	Provider stack

If asking "does ExamBuddy have classical vector RAG?": No.

If asking "does it retrieve document chunks into the LLM prompt?": Yes - keyword retrieval.

6. DOCUMENT / PDF PROCESSING

Upload (mobile DocumentPicker)

```
-> POST /api/v1/documents (multipart)
-> DocumentService.upload_document
-> FileService.detect_file_type + upload_file (Cloudinary or local)
-> DocumentRepository.create (status PROCESSING)
-> background _process_document_text
-> extract_and_chunk_async (pdf_processor)
-> extract_text -> preprocess -> chunks
-> update document: extracted_text, text_chunks, page_count, READY
-> if category=syllabus: extract_syllabus_structure -> syllabus_structure
```

Concern	Verified detail
PDF parser	PyMuPDF
OCR	pytesseract path for images / hard PDFs (when available)
DOCX	python-docx
File storage	Cloudinary if CLOUDINARY_CLOUD_NAME set; else backend/uploads/{user_id}/...
Size limit	MAX_UPLOAD_SIZE_MB default 20 (config.py)

Formats	pdf,docx,png,jpg,jpeg (ALLOWED_FILE_TYPES)
Extracted text storage	MongoDB documents.extracted_text
Original files	Stored (Cloudinary URL or local path / local://)
Chunking	Yes - text_chunks on document
Sent directly to LLM?	Analysis uses cleaned extracted content (bounded/processed). Notes use retrieved chunks + PYQ context, not necessarily entire raw PDF every time.

7. SYLLABUS PROCESSING

Upload with category syllabus

```
-> same extraction as documents
-> extract_syllabus_structure (syllabus_parser.py)
-> subjects / units(modules) / topics / subtopics + flat catalog
-> stored on document
-> later: build_module_topic_tree (syllabus_modules.py)
    Subject -> Module -> Topic
    PYQ topics matched (threshold 0.82)
    unmatched PYQ -> "Other / Unmapped Topics"
```

Question	Answer from code
How modules detected	Heuristic regex (MODULE I/II, Unit, Chapter, etc.)
How topics detected	Line/structure heuristics under modules
LLM involved?	No in parser
Storage	On syllabus document (syllabus_structure) + used in subjects overview aggregation
Match with PYQ	SequenceMatcher / similarity; assign if score >= 0.82

8. PYQ ANALYSIS PIPELINE

PYQ Upload (category pyq)

```
-> text extraction + chunks stored
-> POST /analysis/pyq (document ids)
-> AnalysisService.create_analysis -> background _run_analysis
-> use stored extracted_text
-> NotesPipeline / cleaning
-> rule-based topic_extractor.extract_topics / filter_topics
-> AIService.analyze_pyq (LLM JSON enrich) OR local fallback
-> sanitize / priority fields
-> optional match_topics_to_syllabus
-> save pyq_analyses
-> mobile AnalysisResult
```

Topic extraction method

Hybrid:

1. Primary extraction: rule-based (topic_extractor.py) - regex/heuristics, not embeddings.
2. LLM enrichment: validate/merge into canonical topic frequency JSON (PYQ_ANALYSIS_* prompts).
3. Syllabus mapping: string similarity threshold 0.82 (not embedding-based).

Exact system prompt: see Section 3 / backend/app/services/ai/prompts.py PYQ_ANALYSIS_SYSTEM_PROMPT.

Question number / marks extraction as a fully separate structured parser stage: partially present in cleaning/pipeline utilities; dedicated perfect question-segmentation engine is limited - rely on text cleaning + topic heuristics. Exact universal marks parser completeness: cannot be fully verified as complete for all paper formats.

9. TOPIC NORMALIZATION

Implemented in multiple places:

Mechanism	File	Behavior
normalize_topic_key	generated_notes_repository.py	Cache key normalization
canonical_topic_key	topic_priority.py	Lowercase, strip punctuation/hyphen variants, collapse spaces
resolve_canonical_name	topic_priority.py	Prefer syllabus name if similarity ≥ 0.82
_similarity	topic_priority.py	Exact, substring (≥ 4 chars $\rightarrow 0.93$), else SequenceMatcher.ratio()
LLM merge instruction	PYQ_ANALYSIS_SYSTEM_PROMPT	"Map similar concepts to ONE standardized topic name"

Example: "Black Box Testing" vs "Black-box testing" vs "Black Box Test"

- Canonical keys strip punctuation/hyphens and normalize spaces $\rightarrow$ often collide to the same key.
- If a syllabus topic is close (≥ 0.82), the syllabus spelling becomes canonical.
- They are intended to be treated as the same topic for aggregation/caching when keys match; edge cases with very different wording may remain separate.

Semantic embedding matching: Not found.

Exact matching: Used when keys equal.

Fuzzy matching: SequenceMatcher thresholds (0.82 assign; near-miss band in modules utils).

10. MODULE-WISE SYLLABUS MAPPING

Implemented (verified):

```
Subject
-> Module (unit/chapter from syllabus parse)
-> Topic (syllabus topics + matched PYQ occurrence/priority)
Unmapped PYQ topics -> "Other / Unmapped Topics"
```

Files:

- backend/app/utils/syllabus_parser.py
- backend/app/utils/syllabus_modules.py (build_module_topic_tree, _MATCH_THRESHOLD = 0.82)
- Subjects overview API feeds mobile SubjectNotes $\rightarrow$ ModuleTopics $\rightarrow$ TopicStudyNotes

Unmapped handling: bucket module "Other / Unmapped Topics"; near-miss 0.62-0.81 can be flagged needs_review without hard assign (per syllabus_modules.py logic).

11. PYQ OCCURRENCE & PRIORITY

Implemented in backend/app/utils/topic_priority.py.

Occurrence

Aggregates topic frequency / paper counts across analyses for a subject (via aggregation helpers in the same module / subject service).

Priority score (actual logic)

```
occurrence_score = (frequency / max_frequency) * 100
```

```

if marks available:
    priority_score = 0.7 * occurrence_score + 0.3 * marks_score
else:
    priority_score = occurrence_score

```

Then High/Medium/Low thresholds depend on number of analyzed papers (softer when few papers). Example for ≥ 3 papers:

- High if `priority_score` ≥ 80
- Medium if ≥ 50
- else Low

Special cases: single appearance across many papers forced toward Low.

Marks weightage: used when marks data is available; otherwise occurrence-only.

12. NOTES GENERATION PIPELINE

```

Notes tab -> SubjectNotes -> ModuleTopics -> TopicStudyNotes
-> POST /notes/topic/generate (or /notes/generate, stream, regenerate)
-> NotesService.generate_topic_note
-> cache lookup (user_id, topic_key, analysis_id) if not regenerate
-> hit only if notes exist AND ai_metadata.prompt_version == PROMPT_VERSION ("v15.0")
-> else DocumentRetriever context + PYQ/analysis context
-> AIService.generate_topic_notes (LLM JSON)
-> GeneratedNotesRepository.upsert
-> response to mobile

```

Question	Answer
Which model	Env-selected provider model (see §3)
Context	PYQs related to topic, keyword RAG chunks, analysis signals, exam_priority, optional unit/module
Prompt	TOPIC_NOTES_*
PYQs included?	Yes (as related questions context)
Syllabus included?	Indirectly via unit/module/subject + mapping; full syllabus dump not required every call
Uploaded PDFs?	Via retrieved chunks when matching
Stored?	Yes - generated_notes
Duplicate prevention	Cache by topic_key + analysis_id + prompt_version
On-demand vs bulk	Both: per-topic endpoints and batch /notes/generate

13. QUIZ GENERATION PIPELINE

```

Quiz tab -> subject select -> QuizConfig
-> POST /quiz/generate (QuizService.generate_quiz)
-> AIService.generate_quiz
-> QUIZ_GENERATE_* prompts
-> store quizzes
-> QuizPlay -> submit -> quiz_attempts (+ analysis endpoints)

```

Item	Detail
Model	Same multi-provider LLM stack
Prompt	QUIZ_GENERATE_SYSTEM_PROMPT / USER
Output	questions with type, options, correct_answer, explanation, topic

Difficulty	Request parameter (easy/medium/hard style - passed into prompt)
Fallback	_local_quiz_result

14. BACKEND API MAP

Base prefix: /api/v1 (API_V1_PREFIX). Router: backend/app/api/v1/router.py.

Authentication (auth.py)

Method	Endpoint	Purpose	File
POST	/auth/register	Register	auth.py
POST	/auth/login	Login	auth.py
POST	/auth/refresh	Refresh token	auth.py
POST	/auth/forgot-password	Request reset	auth.py
POST	/auth/reset-password	Reset with token	auth.py
GET	/auth/me	Current user	auth.py

Documents / Files (documents.py)

Method	Endpoint	Purpose	File
POST	/documents	Upload document	documents.py
POST	/documents/... (second upload/helper route in file)	Upload variants	documents.py
GET	/documents	List documents	documents.py
DELETE	/documents	Clear all	documents.py
GET	/documents/{id}	Get document	documents.py
GET	/documents/{id}/status	Processing status	documents.py
PATCH	/documents/{id}	Update metadata	documents.py
DELETE	/documents/{id}	Delete	documents.py

Analysis / PYQ (analysis.py)

Method	Endpoint	Purpose	File
POST	/analysis/pyq	Create analysis	analysis.py
GET	/analysis/pyq	List analyses	analysis.py
GET	/analysis/pyq/{id}	Get result	analysis.py
GET	/analysis/pyq/{id}/status	Status	analysis.py
DELETE	/analysis/pyq/{id}	Delete	analysis.py

Subjects / Topics (subjects.py)

Method	Endpoint	Purpose	File
GET	/subjects	List subjects	subjects.py
GET	/subjects/{id}/topics	Topics	subjects.py
GET	/subjects/{id}/overview	Module/topic overview + priority	subjects.py
DELETE	/subjects/{id}	Remove subject	subjects.py

Notes (notes.py)

Method	Endpoint	Purpose	File
POST	/notes/generate	Batch/single generate	notes.py
POST	/notes/topic/generate	Cached topic notes	notes.py
POST	/notes/topic/regenerate	Force regenerate	notes.py
POST	/notes/topic/stream	SSE stream	notes.py
GET	/notes/topic/status	Cache status	notes.py
GET	/notes/generated	List generated	notes.py
GET	/notes/generated/{id}	Get by id	notes.py
PATCH	/notes/generated/{id}/save	Save/unsave	notes.py

POST	/notes/simplify (in file)	Simplify	notes.py
GET	/notes	List notes	notes.py
DELETE	/notes	Clear notes	notes.py
GET	/notes/{id}/export/pdf	PDF export	notes.py
GET/PATCH/DELETE	/notes/{id}	CRUD	notes.py

Quiz (quiz.py)

Method	Endpoint	Purpose	File
POST	/quiz/generate (in file)	Generate quiz	quiz.py
GET	/quiz	List quizzes	quiz.py
DELETE	/quiz	Clear quizzes	quiz.py
GET	/quiz/{id}	Get quiz	quiz.py
POST	/quiz/{id}/submit	Submit answers	quiz.py
GET	/quiz/{id}/attempts	Attempts	quiz.py
DELETE	/quiz/{id}	Delete quiz	quiz.py
GET	/quiz/history	Attempt history	quiz.py
GET	/quiz/analysis/{subject}	Performance	quiz.py
GET/DELETE	/quiz/attempts/{id}	Attempt detail/delete	quiz.py

User / Profile (profile.py)

Method	Endpoint	Purpose	File
GET	/dashboard	Stats	profile.py
DELETE	/dashboard/activities	Clear activities	profile.py
DELETE	/dashboard/activities/{ref_id}	Delete activity	profile.py
GET/PATCH	/profile	Profile	profile.py
PATCH	/profile/preferences	Preferences	profile.py
PATCH	/profile/password	Change password	profile.py
DELETE	/profile	Delete account	profile.py

Health: /health referenced in render.yaml (app main).

15. FRONTEND ARCHITECTURE

Aspect	Detail
Framework	Expo ~54 + React Native 0.81 + TypeScript

Routing	React Navigation - Root stack + Auth stack + Main tabs
State	Zustand stores (auth, notes, quiz, etc.)
API	axios to backend /api/v1 (production URL configured in EAS / env)
Auth storage	Expo SecureStore for tokens
UI	react-native-paper + custom screens

Navigation flow (actual)

```
Splash
+- Auth
|   +- Login
|   +- Signup
+- Main (tabs)
    +- Dashboard
    +- Notes -> SubjectNotes -> ModuleTopics -> TopicStudyNotes
    +- Quiz -> QuizSubjectSelect -> QuizConfig -> QuizPlay -> QuizResult
      |   (+ QuizHistory, QuizAnalysis, QuizAttemptReview)
    +- Profile

Stack also: UploadPYQ, UploadedDocuments, DocumentViewer, AnalysisResult, NoteDetail, ...
```

Loading/error handling: screen-level + store flags (standard RN patterns). Exact global error boundary framework: not a dedicated Sentry/Crashlytics integration found as required core - treat as app-level handling unless added outside audited files.

16. DATABASE ARCHITECTURE

Technology: MongoDB (MONGODB_URI, DB name smartstudy) via Motor (backend/app/db/mongodb.py).

Collections (verified)

Collection	Purpose
users	Accounts, credentials hash, profile
documents	Uploaded files metadata, extracted_text, text_chunks, syllabus_structure
pyq_analyses	PYQ analysis results
notes	Batch/legacy notes records
generated_notes	Per-topic AI notes cache
subjects	Subject list / metadata
quizzes	Generated quizzes
quiz_attempts	Submissions
quiz_analysis	Aggregated quiz performance
user_stats	Dashboard stats

Indexes: backend/app/db/indexes.py.

Logical relationships (actual)

```
User
+- documents (user_id)
|   +- text_chunks / extracted_text / syllabus_structure
+- pyq_analyses (user_id, document_ids)
+- subjects (derived/listed per user)
+- generated_notes (user_id, topic_key, analysis_id)
+- notes
+- quizzes -> quiz_attempts
```

+ user_stats / quiz_analysis

There is no separate relational Module table; modules live inside syllabus structure / aggregated overview payloads.

17. FILE STORAGE

Mode	When	Path/URL
Cloudinary	CLOUDINARY_CLOUD_NAME (and keys) set	Cloudinary upload; folder default smartstudy/documents
Local	Cloudinary not configured	backend/uploads/{user_id}/{uuid}_{filename}; URL scheme local://...

Retrieval: FileService downloads local path or HTTP GET remote URL.

Deletion: document delete flows remove DB records; Cloudinary/local cleanup depends on FileService delete helpers - verify per delete path when auditing ops.

Security: uploads scoped by authenticated user_id; JWT required on document routes.

18. AUTHENTICATION & SECURITY

Topic	Implementation
Mechanism	Email/password -> JWT access + refresh
Password	bcrypt via passlib (get_password_hash / verify_password)
JWT	HS256; JWT_SECRET; access ~30 min; refresh ~7 days
Token storage (mobile)	Expo SecureStore
Protected routes	FastAPI Depends(get_current_user)
Data isolation	Queries filter by user_id / current user id
File access	Authenticated document APIs
Secrets	Env vars only - values not documented here

Variable names (no values): JWT_SECRET, MONGODB_URI, GROQ_API_KEY, OPENAI_API_KEY, GEMINI_API_KEY, CLOUDINARY_*, SMTP_*.

19. COMPLETE END-TO-END FLOW

```
Student registers/logs in (Auth API + SecureStore)
-> Uploads syllabus (optional) & PYQs (UploadPYQ / documents API)
-> Backend stores file + extracts text/chunks (and syllabus_structure if syllabus)
-> Student starts PYQ analysis
-> Rule topic extraction + LLM enrichment + syllabus match + priority aggregation
-> Student opens Notes -> picks Subject -> Module -> Topic
-> Notes generate (cache or LLM+keyword RAG) -> read notes
-> Student opens Quiz -> configure -> generate -> play -> submit -> history/analysis
```

Each step maps to files cited in §§6-15.

20. ACTUAL CODE REFERENCES (MAJOR)

File	Symbol	Responsibility
backend/app/main.py	FastAPI app	App entry, router mount, CORS
backend/app/core/config.py	Settings	Env configuration

backend/app/core/security.py	JWT + password helpers	Auth crypto
backend/app/db/mongodb.py	connect_to_mongo	DB connection
backend/app/services/document_service.py	upload_document, _process_document_text	Upload pipeline
backend/app/services/file_service.py	upload_file	Cloudinary/local storage
backend/app/utlis/text_extractor.py	extract_text	PDF/DOCX/OCR
backend/app/utlis/pdf_processor.py	extract_and_chunk_*	Chunk storage prep
backend/app/services/analysis_service.py	create_analysis, _run_analysis	PYQ analysis orchestration
backend/app/utlis/topic_extractor.py	extract_topics	Rule-based topics
backend/app/utlis/syllabus_parser.py	extract_syllabus_structure, match_topics_to_syllabus	Syllabus parse/match
backend/app/utlis/syllabus_modules.py	build_module_topic_tree	Module-wise tree
backend/app/utlis/topic_priority.py	classify_priority, aggregations	Occurrence/priority
backend/app/services/rag/retriever.py	DocumentRetriever	Keyword chunk retrieval
backend/app/services/ai/ai_service.py	analyze_pyq, generate_topic_notes, generate_quiz	LLM facade
backend/app/services/ai/prompts.py	prompt constants	Prompt source of truth
backend/app/services/notes_service.py	generate_topic_note	Notes + cache
backend/app/api/v1/*.py	routers	HTTP API
mobile/src/navigation/*	navigators	App structure

21. ENVIRONMENT VARIABLES

VARIABLE_NAME | Purpose | Used in

APP_NAME | App title | config.py / .env.example

APP_VERSION | Version | config.py

ENVIRONMENT | dev/staging/production | config.py, render.yaml

DEBUG | Debug flag | config.py

API_V1_PREFIX | API prefix | config.py

MONGODB_URI | Mongo connection | config.py, mongodb.py

MONGODB_DB_NAME | DB name (smartstudy) | config.py

JWT_SECRET | JWT signing | security.py / settings

JWT_ALGORITHM | Usually HS256 | settings

ACCESS_TOKEN_EXPIRE_MINUTES | Access TTL | settings

REFRESH_TOKEN_EXPIRE_DAYS | Refresh TTL | settings

CORS_ORIGINS | CORS | settings / main

CLOUDINARY_CLOUD_NAME | Cloudinary | file_service / settings

CLOUDINARY_API_KEY | Cloudinary | settings

CLOUDINARY_API_SECRET | Cloudinary | settings

AI_PROVIDER | groq\ | gemini\ | openai | ai_service / settings

GROQ_API_KEY / GROQ_MODEL | Groq | settings / providers

OPENAI_API_KEY / OPENAI_MODEL | OpenAI | settings / providers

GEMINI_API_KEY / GEMINI_MODEL | Gemini | settings / providers

SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD | Email reset | settings

SMTP_FROM_EMAIL | From address | settings

FRONTEND_RESET_URL | Deep link reset | settings

MAX_UPLOAD_SIZE_MB | Upload limit | settings

ALLOWED_FILE_TYPES | Allowed extensions | settings

Mobile API base URL: configured via Expo/EAS env (e.g. production Render host) - see mobile/eas.json / app config. Exact secret values: not listed.

22. EXTERNAL SERVICES

Service	Role	Evidence
MongoDB Atlas (typical)	Database hosting	MONGODB_URI srv pattern in .env.example
Groq API	LLM (optional)	Groq settings + OpenAI-compatible client usage
Google Gemini API	LLM (optional; Render default provider)	
OpenAI API	LLM (optional)	openai package
Cloudinary	File hosting (optional)	cloudinary package + settings
Render	Backend deploy	render.yaml
Expo EAS	Mobile builds	eas.json, scripts
SMTP provider	Password reset email (optional)	SMTP settings

23. DEPENDENCY AUDIT

Backend (backend/requirements.txt) - important

Dependency	Why
fastapi, uvicorn, python-multipart	API server
motor, pymongo	MongoDB
pydantic, pydantic-settings, email-validator	Validation/settings
python-jose, passlib, bcrypt	Auth
cloudinary, httpx	Storage / HTTP
PyMuPDF, python-docx, Pillow, pytesseract	Document/OCR
openai, google-generativeai	LLM providers
fpdf2	PDF export
python-dotenv	Env loading
pytest, pytest-asyncio	Tests

Mobile (mobile/package.json) - important

Expo, RN, React Navigation, axios, zustand, paper, secure-store, document-picker, file-system, webview, etc.

Notes / caveats (report only - not removed)

- Root-level requirements.txt (if present) may contain scientific packages (numpy/scikit-learn) not clearly wired into the FastAPI AI path - treat as possible unused/legacy at root unless imported by audited backend modules.
- No vector DB / LangChain packages in backend requirements.
- Dual branding SmartStudy vs ExamBuddy in package names is cosmetic inconsistency, not a runtime bug by itself.
- fpdf2 used for export; also suitable tooling for this audit PDF.

24. DEPLOYMENT ARCHITECTURE

Mobile (Expo APK via EAS or local gradle)

- > HTTPS -> FastAPI on Render (Docker: backend/Dockerfile)
 - > MongoDB (Atlas URI)
 - > LLM provider APIs (Gemini default in render.yaml)
 - > Cloudinary (if configured) OR ephemeral/local disk (local mode less ideal on free PaaS)

Evidence: render.yaml (web service smartstudy-api, health /health, env blueprint), backend/Dockerfile, mobile/eas.json.

Frontend web hosting as a separate SPA: Not found as primary - product is mobile-first.

25. PERFORMANCE & COST (RECOMMENDATIONS ONLY)

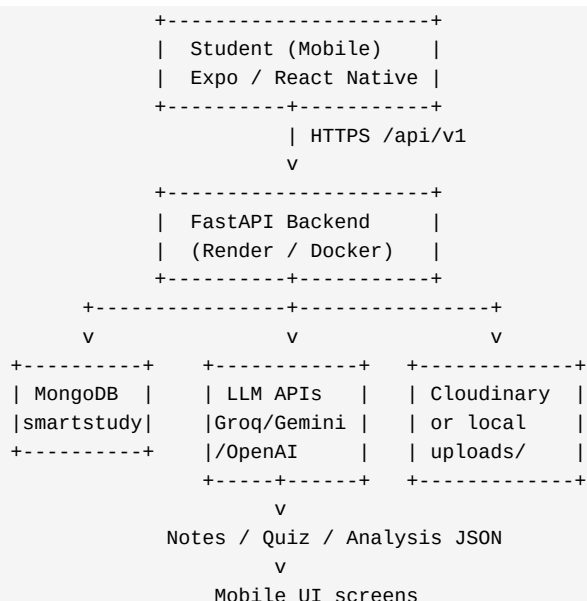
Expensive operations observed

- LLM calls for analysis, notes, quizzes (token cost + latency).
- PDF extraction/OCR on upload.
- Keyword retrieval scanning multiple documents/chunks.
- Regenerating notes ignoring cache.
- Large content windows into quiz/analysis prompts.

Opportunities (do not implement here)

- Keep/enforce notes cache (prompt_version) - already present; educate UI to avoid regenerate.
- Bound RAG context (already ~8 chunks / 10k chars) - tune further.
- Prefer Gemini/Groq cost tiers intentionally; avoid accidental multi-fallback retries burning multiple providers.
- Background jobs already used for analysis/upload - extend for heavy batch notes.
- Ensure Mongo indexes remain healthy (indexes.py).
- Store/reuse quiz only when needed; clear old quizzes.
- OCR only when native text extraction is empty.

26. CURRENT ARCHITECTURE DIAGRAM



27. RAG ARCHITECTURE DIAGRAM (ACTUAL)

Vector RAG: not implemented. Actual alternative:

```
Documents (PDF/DOCX/Images)
-> Text extraction (PyMuPDF / docx / OCR)
-> Cleaning / preprocess
-> Chunking (1200 / overlap 150)
-> Store text_chunks on MongoDB documents
-> On notes request: keyword/term scoring + category boost
-> Top chunks (max ~8)
-> Prompt {rag_context} + PYQ + topic metadata
-> LLM (multi-provider)
-> Structured notes JSON
-> generated_notes cache
```

28. "HOW EXAMBUDDY WORKS" - SIMPLE EXPLANATION

What is an LLM? A large language model is a remote AI text engine. You send instructions + context; it returns text (here, usually JSON).

Which LLM does ExamBuddy use? Configurable: Groq llama-3.3-70b-versatile, Gemini gemini-2.5-flash, or OpenAI gpt-4o-mini. Render blueprint defaults toward Gemini. If LLMs fail, rule-based local fallbacks exist.

What does the LLM receive? System/user prompts from prompts.py plus topic, subject, PYQ snippets, and retrieved study-material chunks.

What is a prompt? The written instruction telling the model how to behave and what JSON shape to return.

What is RAG? Retrieval-Augmented Generation: find relevant document pieces, put them in the prompt, then generate.

Does ExamBuddy use RAG? It uses a keyword retrieval form of RAG for notes - not embedding/vector-DB RAG.

What are embeddings? Numeric vectors representing meaning. ExamBuddy does not use embeddings for retrieval in the current code.

What is a vector database? A store optimized for vector similarity search. Not found in this codebase.

How does PDF text reach the LLM? Upload -> extract -> store -> (for notes) score chunks by topic words -> insert into prompt -> LLM.

How are topics extracted? Mostly rules/regex; LLM helps normalize/enrich during analysis.

How are syllabus topics matched? Fuzzy string similarity (≥ 0.82), not vectors.

How are notes generated? Cached if possible; else LLM with PYQ + retrieved chunks.

Where are notes stored? MongoDB generated_notes.

How does the frontend get notes? Authenticated HTTP calls to /notes/topic/generate (or related endpoints), then displays on TopicStudyNotes.

29. WHAT IS ACTUALLY IMPLEMENTED VS PLANNED

CURRENTLY IMPLEMENTED (verified)

- Mobile auth + JWT session storage
- Document upload/list/delete + processing status
- PDF/DOCX/image text extraction; optional OCR
- Syllabus heuristic parse + module-wise topic tree

- PYQ analysis with rule topics + LLM enrichment + local fallback
- Topic occurrence & relative High/Medium/Low priority
- Syllabus?PYQ fuzzy mapping (0.82)
- Keyword chunk retrieval into notes prompts
- Topic notes generation, cache, regenerate, stream, PDF export
- Quiz generate/submit/history/analysis
- Profile/dashboard
- Docker/Render deployment config; EAS mobile build scripts
- Multi-provider AI with fallback

NOT CURRENTLY IMPLEMENTED / NOT FOUND

- Vector embeddings pipeline
- Vector database (FAISS/Chroma/Pinecone/Weaviate/Qdrant/pgvector)
- LangChain / LlamaIndex orchestration
- Anthropic Claude provider integration
- On-device mobile LLM inference
- True semantic embedding-based topic matching
- Guaranteed complete marks/question-number extraction for all paper layouts
- Separate production "notes upload as primary knowledge base" product path beyond document categories already supported
- Any feature only mentioned in comments/plans without code path - treat as Not found in the current codebase unless a concrete module exists

30. POTENTIAL PROBLEMS (OBSERVATIONS + SUGGESTIONS)

Problem	Why it matters	Suggestion (not implemented)
Keyword RAG ? semantic RAG	Misses paraphrased material	Optional embeddings later
LLM hallucination	Notes may invent details if context thin	Stronger grounding + refusal rules (partially in prompts)
Large prompt content	Cost/latency	Keep caps; summarize chunks
Provider fallback storms	Multiple paid calls on failure	Circuit breakers / single-provider prod
Fuzzy topic merges wrong	Near-miss topics collide or miss	Human review flag already partially present
Local disk on Render	Ephemeral filesystem risk	Prefer Cloudinary in production
Dual naming SmartStudy/ExamBuddy	Confusion	Align branding gradually
OCR dependency	Needs Tesseract installed in runtime image	Ensure Docker image includes binaries if OCR required
Prompt version cache	Old notes until regenerate	Already versioned - bump PROMPT_VERSION carefully
Security of CORS *	Broad origins	Tighten for production if browser clients expand

31. FINAL SUMMARY

1. Tech stack: Expo/RN mobile + FastAPI + MongoDB + optional Cloudinary + multi-LLM.
2. LLM(s): Groq Llama 3.3 70B / Gemini 2.5 Flash / OpenAI GPT-4o-mini (env-selected; Render blueprint favors Gemini).
3. RAG status: Keyword chunk retrieval into prompts - no vector RAG.
4. Database: MongoDB smartstudy (10 collections listed above).

5. File storage: Cloudinary or local uploads/.
6. Main AI pipelines: PYQ analysis, topic notes, batch notes, simplify, quiz.
7. Main APIs: /auth, /documents, /analysis/pyq, /subjects, /notes, /quiz, /profile//dashboard.
8. Main entities: users, documents, pyq_analyses, subjects, generated_notes, notes, quizzes, quiz_attempts, quiz_analysis, user_stats.
9. Limitations: No embeddings/vector DB; heuristic syllabus/PYQ matching; LLM cost; OCR/runtime deps; branding inconsistency.
10. Recommended improvements: Production Cloudinary; tighter CORS; optional true vector RAG; stronger evaluation of topic matching; cost controls on fallbacks; keep cache discipline.

End of audit documentation. All claims above are based on repository inspection as of the audit date. Items without code evidence are marked as not found / cannot be verified.